

Agentic AI, RAG, Memory, Planning and Agent Platform Deployment

Discussion Summary for Solution Architecture Reference

Prepared for	Sachin Joshi, Senior Architect
Purpose	Consolidated PDF covering all topics discussed in the thread
Date	15 Jul 2026
Scope	RAG, agents, reasoning loop, memory, planning, deterministic control, out-of-the-box thinking, and deployment platforms

Core idea: RAG helps an LLM know enterprise context. Agents help an LLM reason, plan, use tools, act, observe outcomes, remember state, and continue until a goal is achieved.

1. Executive Summary

This document summarizes the complete discussion thread from a solution architect perspective. It explains how Retrieval Augmented Generation (RAG), agents, memory, planning, MCP, vector databases, and deployment platforms fit together in enterprise GenAI systems.

- RAG is a knowledge-retrieval pattern: it retrieves relevant enterprise content and injects it into the LLM prompt.
- An agent is a goal-oriented AI worker: it can plan, choose tools, execute actions, observe results, update memory, and continue iterating.
- The reasoning loop is the agent execution cycle: Reason -> Act -> Observe -> Update Memory -> Reason Again.
- Memory is the agent state mechanism: it stores working facts, conversation context, long-term preferences, and retrieved knowledge references.
- Planning is often non-deterministic because LLMs generate probable next steps, not fixed algorithmic paths.
- Enterprise-grade deployments should use hybrid architecture: deterministic workflows for high-risk operations and LLM planning for adaptive analysis.
- Agent platform selection should focus on identity, governance, observability, data boundaries, cost control, RAG integration, tool calling, and vendor lock-in.

2. RAG - Retrieval Augmented Generation

RAG allows an LLM to answer using enterprise data rather than relying only on pre-trained model knowledge. Instead of retraining the model for every document change, the architecture retrieves relevant content at query time and passes it to the LLM as grounding context.

Without RAG:

User -> LLM -> Answer

With RAG:

User -> Retrieval System -> Relevant Documents

User + Retrieved Context -> LLM -> Grounded Answer

Why RAG matters

- Improves factual grounding by using current enterprise knowledge.
- Reduces hallucination risk by constraining answers to retrieved context.
- Supports auditability because answers can cite source documents, pages, chunks, or records.
- Avoids expensive retraining for frequently changing business knowledge.

Typical RAG use case

Example: A user asks, "What is our maternity leave policy?" The LLM may not know the company-specific policy. RAG retrieves the relevant HR policy chunk, injects it into the prompt, and the LLM answers based on that document.

3. RAG Components and How They Relate

A production RAG system is not one component. It is a pipeline of ingestion, transformation, indexing, retrieval, prompt construction, and generation.

Component map

Component	Purpose	Architectural Notes
Data Sources	Provide enterprise knowledge such as PDFs, Word docs, SharePoint, Confluence, Jira, websites, APIs, databases.	Must be governed by ownership, classification, freshness, and access policy.
Ingestion Service	Extracts text and metadata from source systems.	Handles file parsing, permissions, versioning, and change detection.
Chunking Service	Splits large documents into smaller meaningful sections.	Good chunking improves retrieval quality and reduces prompt noise.
Embedding Model	Converts text into numeric vectors representing semantic meaning.	The same or compatible embedding model should be used for document chunks and user queries.
Vector Database	Stores embeddings and metadata, then performs nearest-neighbor similarity search.	Examples include Vertex AI Vector Search, Azure AI Search, Amazon OpenSearch, Pinecone, Weaviate, Qdrant, Milvus.
Retrieval Service	Finds top-k chunks relevant to the user query.	May combine vector search, keyword search, filters, reranking, and access checks.
Prompt Builder	Combines system instruction, user question, retrieved chunks, and constraints.	Critical for grounding, citation behavior, and answer quality.
LLM	Generates the final answer using the prompt and retrieved context.	Could be GPT, Gemini, Claude, Llama, or another model.

Chunking

Chunking splits long documents into smaller units, for example sections, paragraphs, pages, or semantic blocks. Embeddings work better on focused pieces of text rather than large mixed-topic documents.

```
Large HR Policy Document
-> Chunk 1: Vacation policy
-> Chunk 2: Maternity policy
-> Chunk 3: Medical policy
```

User asks about maternity leave. Retrieval should return Chunk 2, not the entire document.

Embeddings

Embeddings convert natural language into numeric vectors. Words with similar meaning produce vectors that are close to each other. This allows semantic search rather than only keyword matching.

```
"Car"      -> [0.10, 0.40, 0.20, ...]
"Vehicle"  -> [0.10, 0.41, 0.21, ...]
"Automobile"-> [0.11, 0.39, 0.19, ...]
```

Vector database record

```
{
  "id": "chunk-123",
  "text": "Employees are entitled to six months maternity leave...",
  "embedding": [0.12, 0.55, -0.88, ...],
  "metadata": {
    "source": "HRPolicy.pdf",
    "page": 12,
    "accessGroup": "HR-Employees"
  }
}
```

RAG flow

Documents -> Ingestion -> Chunking -> Embeddings -> Vector DB

User Question -> Query Embedding -> Similarity Search -> Top Chunks
 Top Chunks + User Question -> Prompt -> LLM -> Grounded Answer

4. MCP and RAG

RAG and MCP solve different problems. RAG retrieves knowledge. MCP, or Model Context Protocol, standardizes how models and agents connect to tools and enterprise systems.

Concept	Primary Purpose	Example
RAG	Read and retrieve knowledge.	Search architecture documents, policies, wiki pages, product manuals.
MCP	Connect the model or agent to tools and systems.	Create Jira ticket, query ServiceNow, call GitHub, execute database query through a controlled tool.
Agent	Orchestrate reasoning, RAG, tools, memory, and actions.	Diagnose an issue, retrieve docs, call APIs, create ticket, and summarize evidence.

Useful memory aid:

RAG = Know
MCP = Do
Agent = Orchestrate knowing and doing

5. What Is an Agent?

An agent is a goal-oriented AI worker that uses an LLM plus memory, tools, and a control loop to complete a task. A chatbot usually answers once. An agent can plan, act, observe, and continue until the goal is achieved.

Chatbot:
Question -> LLM -> Answer

Agent:
Goal -> Plan -> Use Tools -> Observe Results -> Update Memory -> Continue -> Outcome

Core agent components

Component	Role
LLM	Reasoning and language generation brain.
Memory	Stores working state, conversation context, and long-term facts.
Tools	External capabilities such as APIs, databases, search, email, Jira, GitHub, or RAG.
Planner	Breaks a goal into steps and chooses the next action.
Executor	Runs tool calls and workflows safely.
Evaluator / Verifier	Checks whether the result is correct, sufficient, safe, and compliant.

Agent relevance to RAG

In modern systems, RAG is commonly one tool available to an agent. The agent decides when it needs knowledge, when it needs live enterprise data, and when it needs to perform an action.

User -> Agent
-> RAG Search for knowledge
-> API / MCP Tool for live data or action
-> LLM synthesis
-> Final response or executed workflow

6. Example - Used Car Dealer SaaS

The discussion used a vehicle marketplace scenario to clarify the difference between RAG and an agent.

Question

Why is vehicle MH12AB1234 not visible in search?

RAG-only answer

- Search documentation about vehicle approval and search indexing.
- Return a general explanation of the event-driven indexing flow.

Agent answer

- Call Vehicle Service: confirm vehicle exists and status is APPROVED.
- Call Search Service: check whether the vehicle is indexed.
- Call Event Store: check whether VehicleApproved event was published.
- Use RAG to retrieve architecture documentation only if workflow knowledge is needed.
- Conclude root cause based on evidence, such as missing event or failed indexing.

Vehicle Status = APPROVED
Search Index = Missing
VehicleApproved Event = Missing

Likely root cause: approval event was not published, so the search service never indexed the vehicle.
Recommended action: replay or republish the VehicleApproved event and monitor indexing completion.

7. Reasoning Loop in Agents

The reasoning loop is the mechanism that makes an agent different from a simple chatbot. It is the iterative cycle the agent follows to achieve a goal.

Reason -> Act -> Observe -> Update Memory -> Reason Again -> Continue until goal is reached

Step-by-step loop

Step	What Happens	Example
Reason	Agent decides what information is needed and what action is best.	Need vehicle status first.
Act	Agent invokes a tool or retrieval action.	GET /vehicles/MH12AB1234
Observe	Agent reads the result and updates its understanding.	Vehicle is APPROVED.
Reason Again	Agent uses updated facts to choose the next action.	Now check search index.
Stop / Continue	Agent stops when success criteria are met, otherwise loops.	Root cause found or data unavailable.

Agentic RAG

Traditional RAG often retrieves once and generates once. Agentic RAG can retrieve multiple times, call APIs, compare evidence, and refine the query if the first retrieval is insufficient.

Question -> Agent -> Search Docs
Enough context? No -> Search again or call API
Need live data? Yes -> Tool call

Evidence sufficient? Yes -> Generate final answer

8. How Memory Affects the Reasoning Loop

Memory allows each loop to build on previous observations. Without memory, the agent would repeatedly rediscover the same facts or lose important context.

Types of memory

Memory Type	Scope	Typical Store	Example
Working Memory	Current task execution.	In-memory state, Redis, workflow state.	Vehicle approved, search index missing, event missing.
Conversation Memory	Current user session.	Session store, conversation history.	User says "create ticket for it" and agent knows what "it" means.
Long-Term Memory	Across sessions.	Database, profile store, secure memory service.	Preferred region, reporting style, frequent entities.
Knowledge Memory / RAG	External enterprise knowledge.	Vector DB plus document store.	Policies, runbooks, product docs, architecture standards.

Memory update example

Loop 1 observation: Vehicle Status = APPROVED
Working memory: [Vehicle Approved]

Loop 2 observation: Search Index = Missing
Working memory: [Vehicle Approved, Not Indexed]

Loop 3 observation: VehicleApproved Event = Missing
Working memory: [Vehicle Approved, Not Indexed, Event Missing]

Conclusion: Missing event caused indexing failure.

Context window and memory management

LLMs have finite context windows. As memory grows, agents must summarize, compress, and retrieve relevant memories on demand. This is similar to RAG, but applied to agent state and previous observations.

- Summarize long histories into key findings.
- Store detailed history outside the prompt.
- Retrieve only relevant memories for the next reasoning step.
- Protect memory using tenant isolation, encryption, retention policy, and access controls.

9. Agent Planning

Planning is the process of converting a user goal into executable steps. The agent considers the goal, available tools, memory, retrieved knowledge, rules, and constraints.

Goal -> Understand intent -> Identify available tools -> Create plan -> Execute step -> Observe -> Re-plan if needed

Example plan

Goal: Generate dealer performance report

1. Get sales data
2. Aggregate by dealer
3. Calculate rankings
4. Generate PDF

Why planning is non-deterministic

LLMs generate likely next tokens rather than executing fixed algorithms. Even for the same goal, an LLM can create multiple valid plans. The variation may come from ambiguous goals, probabilistic token generation, different tool choices, and different intermediate reasoning paths.

Source of Non-Determinism	Impact
Probabilistic generation	Small differences in token choices can change the plan.
Ambiguous goals	The agent may choose different interpretations.
Multiple valid tools	The agent may choose API, database, RAG, or logs differently.
Iterative reasoning	A small early difference can lead to a different execution path.

10. Improving Agent Planning Quality

Planning quality improves when the LLM is not left to freely improvise. Give it goals, tool descriptions, cost metadata, examples, runbooks, constraints, and validation rules.

Practical measures

- Use clear goals: specify success criteria, expected output, and constraints.
- Define tools clearly: include purpose, input schema, output schema, side effects, cost, latency, and when not to use.
- Use planning templates for common patterns such as incident diagnosis, eligibility checks, report generation, and data analysis.
- Use Retrieval-Augmented Planning: retrieve SOPs, runbooks, and previous successful plans before planning.
- Generate structured plans, preferably JSON, so they can be validated and audited.
- Add stop conditions to prevent unnecessary loops and tool calls.
- Add cost-aware planning: prefer memory, cache, and targeted APIs before expensive broad searches or logs.
- Use plan validators to enforce security, compliance, tool usage, and approval rules.
- Use reflection: ask the agent to evaluate whether the plan is sufficient before execution.

Structured plan example

```
{
  "goal": "Diagnose why vehicle is not searchable",
  "constraints": ["minimize tool calls", "avoid broad log scans", "do not perform write actions"],
  "steps": [
    {"action": "getVehicleById", "reason": "confirm status"},
    {"action": "getSearchIndexStatus", "condition": "status is APPROVED or ACTIVE"},
    {"action": "getVehicleEvents", "condition": "vehicle not indexed"}
  ],
  "stop_condition": "root cause identified with evidence or required data unavailable"
}
```

11. Can LLM Planning Be Better Than Deterministic Planning?

Yes, in exploratory and context-dependent scenarios, an LLM-assisted planner can produce a more efficient plan than a fixed deterministic workflow. However, for regulated or transaction-critical processes, deterministic workflows remain safer and more auditable.

Scenario	Better Fit	Reason
Root-cause analysis	LLM planner with constraints	Can adapt based on observations and stop early.
Incident troubleshooting	LLM planner plus runbooks	Can generate hypotheses and query only relevant systems.
Business analytics exploration	LLM planner	Can combine signals from multiple domains.
Loan approval	Deterministic workflow	Requires consistent, auditable, policy-driven sequence.
Refund or payment action	Deterministic workflow plus human approval	High financial and compliance risk.

Recommended hybrid model

LLM proposes plan
 Workflow engine constrains execution
 Policy engine validates rules
 Human approves high-risk actions
 Executor performs controlled tool calls
 Observability records every step

12. Determinism in Agentic Systems

Fully deterministic behavior is difficult when the LLM is responsible for planning. But architects can make agent systems highly predictable by separating flexible reasoning from controlled execution.

Controls for deterministic or near-deterministic behavior

- Set model temperature close to 0 to reduce randomness.
- Use fixed workflows or state machines for critical business flows.
- Constrain the LLM with explicit planning rules and allowed tools.
- Require structured output and schema validation.
- Use plan approval before action execution.
- Use human-in-the-loop for destructive, financial, regulated, or irreversible actions.
- Log prompts, plans, tool calls, observations, final outputs, and approvals.

Good architectural rule:

More freedom -> more adaptability but less predictability
 More rules -> more predictability but less flexibility
 Enterprise systems need the right balance.

13. Can LLMs Think Out-of-the-Box?

An LLM can appear to think out-of-the-box by synthesizing patterns across many domains, generating alternative approaches, and combining ideas that may not be obvious from a single deterministic workflow. It does not have conscious intuition or independent goals. Its strength is broad pattern synthesis and rapid hypothesis generation.

Where LLMs can be strong

- Exploring a large solution space quickly.
- Combining ideas across domains, such as marketplace design, CRM, fraud detection, search, event-driven architecture, and AI assistants.

- Generating multiple architecture options such as conservative, cost-optimized, high-scale, AI-native, serverless, or event-driven alternatives.
- Creating hypotheses for production issues from logs, metrics, runbooks, and previous incidents.

How to encourage more creative planning

- Ask for multiple fundamentally different approaches.
- Ask for conservative, startup-style, high-scale, low-cost, and AI-native options.
- Use contrarian prompts such as "avoid the obvious approach" or "challenge this architecture".
- Use multi-agent debate: architect agent, security agent, cost agent, operations agent, and supervisor agent.
- Use reflection: critique the first plan and create a better version.

The strongest enterprise pattern is not replacing the architect. It is using the architect to define goals, constraints, and success criteria while the LLM explores more possible paths.

14. Top 5 Platforms for Agent Deployment

The discussion identified five major platform choices for agent deployment. The best choice depends on the enterprise cloud footprint, identity model, governance needs, data platform, model strategy, and required level of portability.

Platform	Best Fit	Key Strengths	Watchouts
Microsoft Azure AI Foundry Agent Service	Microsoft-heavy enterprises, M365, GitHub, Entra ID, governance-heavy organizations.	Enterprise identity, Azure OpenAI, M365 and Power Platform alignment, strong governance.	Azure-centric architecture and ecosystem dependency.
AWS Bedrock Agents / AgentCore	AWS-first enterprises and multi-model GenAI strategies.	Model choice, Knowledge Bases, Guardrails, strong AWS security integration, modular agent primitives.	AWS ecosystem dependency; verify regional availability and feature maturity.
Google Vertex AI Agent Builder / Agent Engine	GCP-first, data-heavy, analytics-heavy, Gemini and BigQuery-centered environments.	Vertex AI, Gemini, BigQuery, ML tooling, observability, ADK ecosystem.	GCP-centric design; validate connector and enterprise governance requirements.
OpenAI Agents Platform	Startups, rapid prototyping, greenfield AI applications.	Fast development, strong reasoning models, native tool usage, broad developer adoption.	May require additional enterprise controls for large regulated deployments.
LangGraph Platform	Complex workflows, multi-agent orchestration, vendor independence.	Explicit graph/state control, human approval, multi-model support, portability.	More engineering responsibility compared with fully managed cloud-native platforms.

Note: Platform capabilities evolve quickly. Validate current regional availability, pricing, compliance scope, and GA versus preview status before final platform selection.

15. Platform Selection Considerations

Agent platforms should be evaluated based on production-readiness criteria, not only model quality. The following considerations are the most important for enterprise deployment.

1. Identity and access control

- How does an agent authenticate?
- Can the platform integrate with enterprise identity such as Entra ID, IAM, or Google Cloud IAM?

- Can every agent have a unique identity, scoped permissions, and audited access?

2. Data boundaries and tenant isolation

- Where are prompts, memory, traces, and embeddings stored?
- Can data remain in the required region such as India when needed?
- How is tenant isolation enforced in a multi-tenant SaaS platform?

3. RAG integration

- Does the platform support ingestion, chunking, embedding, vector storage, reranking, metadata filtering, and citation handling?
- Can it connect to SharePoint, Confluence, Jira, databases, APIs, file stores, and web sources?

4. Tool calling and MCP

- How are internal APIs registered as tools?
- Are schemas, permissions, retries, timeouts, and side effects managed?
- Does the platform support MCP or equivalent tool protocol patterns?

5. Memory management

- Does the platform support short-term, session, long-term, and vector memory?
- Can memory be encrypted, expired, corrected, and deleted?
- Can memory be segmented by user, tenant, role, and business domain?

6. Observability

- Can teams trace each prompt, plan, tool call, observation, retry, and final answer?
- Can logs be connected to enterprise observability platforms?
- Can architects explain why the agent chose a specific tool or path?

7. Cost control

- Can cost be measured per conversation, per tenant, per workflow, per model, and per tool?
- Can the platform enforce budgets, model routing, caching, token limits, and fallback policies?

8. Governance and safety

- Support for guardrails, content safety, prompt injection defense, data-loss prevention, policy checks, approval workflows, and audit trails.

9. Human approval

- Required for financial, destructive, regulated, external-facing, or irreversible actions.

10. Vendor lock-in and model portability

- Can the architecture switch between GPT, Claude, Gemini, Llama, and other models without rewriting the entire platform?

16. Suggested Architecture for Dealer SaaS Agent Platform

For a GCP-oriented used car dealer SaaS platform, a strong architecture would combine managed GCP AI services with a portable orchestration layer if multi-model flexibility is important.

Dealer User / Admin
 -> API Gateway / BFF
 -> Agent Orchestrator

- > Vertex AI / Gemini for reasoning
- > Vertex AI Vector Search for RAG
- > Dealer Service API
- > Vehicle Service API
- > Search Service API
- > Event Store / Pub/Sub / Logs
- > Policy and Guardrail Service
- > Memory Store by tenant
- > Observability and Cost Tracking
- > Final Answer / Recommendation / Approved Action

Recommended design principles

- Use tenant-aware memory and vector indexes or strict metadata filtering.
- Expose services as safe tools with least-privilege scopes.
- Use deterministic workflows for financing approval, refunds, user deletion, and other high-risk actions.
- Use LLM planning for support diagnostics, dealer insights, search troubleshooting, and architecture assistant scenarios.
- Track cost per dealer, per conversation, and per workflow.
- Store evidence for every recommendation, especially when the agent explains why a vehicle is not searchable or why a dealer is at risk.

17. Key Takeaways

- RAG is for grounding AI answers in enterprise knowledge.
- Agents are for achieving goals using reasoning, memory, tools, and controlled execution.
- MCP and tools allow agents to act, not just answer.
- Memory makes the reasoning loop cumulative and context-aware.
- Planning is non-deterministic but can be improved with templates, rules, retrieval, structured output, validation, and cost-aware constraints.
- LLMs can generate creative alternatives, but enterprise architecture must define the boundaries.
- Hybrid architecture is the safest pattern: deterministic control for critical workflows and adaptive LLM planning for exploratory work.
- Platform choice should prioritize security, governance, observability, RAG, memory, tool integration, cost control, and model portability.

18. Glossary

Term	Meaning
RAG	Retrieval Augmented Generation, a pattern that retrieves external knowledge and injects it into the LLM prompt.
Chunk	A smaller section of a document used for embedding and retrieval.
Embedding	Numeric vector representation of text meaning.
Vector Database	Database optimized for similarity search over embeddings.
Retrieval	Finding the most relevant chunks for a query.
LLM	Large language model used for reasoning and generation.
Agent	Goal-oriented AI system that plans, uses tools, observes results, and iterates.
Reasoning Loop	The Reason -> Act -> Observe cycle used by agents.
Memory	State retained by the agent across steps, sessions, or systems.

Term	Meaning
MCP	Model Context Protocol, a standard pattern for connecting AI systems to tools and data sources.
Tool Calling	Capability for an LLM or agent to invoke external APIs or functions.
Deterministic Workflow	Fixed, predictable process usually implemented with state machines or workflow engines.
Agentic RAG	RAG combined with an agent loop that can retrieve, reason, use tools, and re-plan.

19. Sources and Validation Notes

This PDF is primarily based on the discussion thread. The platform section was also informed by the web research results used during the conversation, including comparative search results about Azure AI Foundry, AWS Bedrock AgentCore, Google Vertex AI Agent Builder, and enterprise agent platforms available in 2026.

- Source references from the conversation: turn8search1, turn8search2, turn8search3, turn8search4, turn8search5.
- Before final platform selection, verify latest official documentation, pricing, regional availability, compliance commitments, and GA versus preview status directly with the cloud provider.